Almalgo Mode · team guide

# The Almalgo agentic loop

How coding agents work in this repo: one entry point, size-gated alignment, nine playbooks, a verification ladder, two-axis review, narrow PRs. Every skill, every flow, every step.

## Contents


01 Overview — The loop at a glance
02 Quick start — Your first task
03 The loop — The size gate
04 Flows — 6 flows, step by step
05 Skill map — How skills load each other
06 Every skill — 28 skill cards
07 Issue states — Triage labels
08 Playbooks — 9 playbooks, every step
09 Verify — The ladder + matrix
10 Principles — 18 principles
11 Autonomy — What needs your OK
12 Files & sources — Where it lives
13 Keep in sync — Updating this guide


Built 2026-10-25 from .agents/ @ 68f2eaf95638 · source: .agents/loop-guide/

Almalgo Mode · `.agents/`

# One loop for every agent task in Almalgo Mode.

28 skills in `.agents/skills/`. Align on the big things, build in small verified slices, review on two axes, ship narrow PRs. One entry point: `/almalgo-mode`.

| 28 | 9 | 3 | 18 |
|---|---|---|---|
| skills | playbooks | verify rungs | principles |

you type it · agent does it · GitHub issue / PR · local stack



# Quick start

Your first task with the loop, in six moves.

1. **Install once per repo**
   curl the kit's install.sh (or clone + run it), then /setup-almalgo-mode. It writes docs/agents/ and the verify ladder.

2. **Open the session**
   In Cursor, type /almalgo-mode and press Alt+Enter (Option+Enter on Mac). It stays on for the whole chat.

3. **Say what you want**
   Paste the issue number or describe the task. The agent sizes it and picks a playbook.

4. **Big or fuzzy?**
   Start with /grill-with-docs instead. Answer its rounds, then /to-spec and /to-tickets.

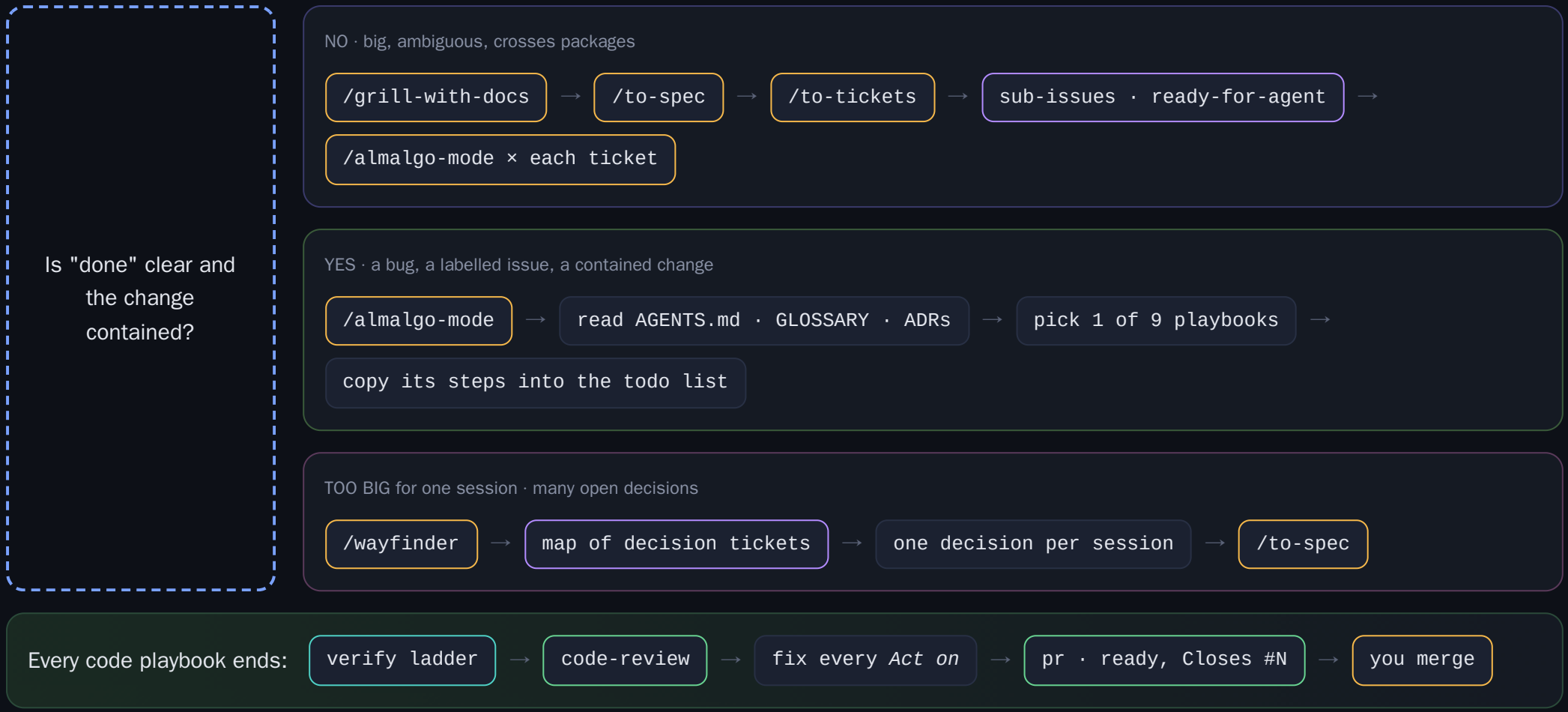
5. **Watch the todo list**
   The playbook steps appear verbatim. A skipped step shows "skip: reason".

6. **Read the evidence**
   Every claim comes with the command that proved it, or a label: inferred or guess.

7. **You merge**
   The agent opens a ready PR. Babysitting and merging happen only when you ask.

## The loop is size-gated

Big work is aligned before code. Small work goes straight in. Both end the same way.



# Flows, step by step

Six flows. Each card is one step; the colour says who does it.

## Big feature

Ambiguous or cross-package work. Align first, then the loop runs once per ticket.

1 · YOU

**/grill-with-docs**

Interview in rounds. GLOSSARY.md + ADRs updated live.

→

2 · YOU

**/to-spec**

Conversation becomes a GitHub spec issue, ready-for-agent.

→

3 · GITHUB

**Spec issue**

Problem, solution, user stories, decisions, test seams.

→

4 · YOU

**/to-tickets**

Vertical slices with blocking edges. You approve granularity.

→

5 · GITHUB

**Sub-issues**

Native blocked-by links. The frontier is visible in GitHub.

→

6 · AGENT

**/almalgo-mode × ticket**

Feature playbook per ticket, or /implement-spec in parallel.

→

7 · LOCAL STACK

**Verify ladder**

docs/agents/verify.md rungs for what the diff touches.

→

8 · AGENT

**code-review → pr**

Fix every Act on. Open ready, Closes #N.

→

9 · YOU

**Babysit / ship**

Only on your ask. Merge is always yours.

## Bug fix

Small and clear: straight into almalgo-mode. Evidence before every line.

1 · GITHUB
**Issue #N**
Labelled ready-for-agent, or you describe it.

→

2 · YOU
**/almalgo-mode**
Sized small, so it picks the Bug fix playbook.

→

3 · AGENT
**diagnosing-bugs**
A loop that goes RED on this bug, on its surface.

→

4 · AGENT
**how · why**
Hypotheses from the subsystem and its history.

→

5 · AGENT
**Root-cause fix**
architect if it crosses a boundary.

→

6 · AGENT
**tdd or red check**
Runner + cheap seam: failing test commit, then fix. Else keep the red check.

→

7 · LOCAL STACK
**Repro → GREEN**
Same surface, plus the verify ladder.

→

8 · AGENT
**code-review → pr**
Red-then-green output pasted in Evidence.

## Parallel spec

/implement-spec: tickets as a graph, the frontier built concurrently.

1 · AGENT
**Integration branch**
One branch for the whole spec.

→

2 · GITHUB
**Ready frontier**
Open tickets with no open blockers.

→

3 · AGENT
**Implementers ×N**
One subagent per ticket, own worktree, tdd.

→

4 · AGENT
**Merger**
Merges each finished ticket into the integration branch.

→

5 · GITHUB
**New frontier**
Unblocked tickets start immediately.

→

6 · AGENT
**code-review**
Once, on the integration branch. One fixer subagent.

→

7 · AGENT
**PR**
Closes the spec and every ticket.

## Triage

Incoming issues become agent-ready briefs or get closed with a reason.

1 · GITHUB
**New issue**
Unlabeled.

→

2 · YOU
**/triage**
"What needs my attention?"

→

3 · AGENT
**Gather + dedupe**
Already implemented? Previously rejected (.out-of-scope)?

→

4 · AGENT
**Recommend**
Category + state, with reasoning. Waits for you.

→

5 · LOCAL STACK
**Verify the claim**
Reproduce the bug from the reporter's steps.

→

6 · AGENT
**Grill if needed**
grilling + domain-modeling.

→

7 · GITHUB
**Outcome**
Agent brief, needs-info notes, or close.

## Wayfinder

Too big for one session. Decisions first, implementation later.

1 · YOU
**/wayfinder idea**
Grill to name the destination.

→

2 · GITHUB
**Map issue**
Destination, decisions so far, fog of war.

→

3 · GITHUB
**Decision tickets**
research · prototype · grilling · task, with blockers.

→

4 · AGENT
**Research ×N**
AFK, in parallel, on throwaway branches.

→

5 · YOU
**One ticket / session**
Claim, resolve, close, record on the map.

→

6 · GITHUB
**Fog graduates**
New tickets as the view clears.

→

7 · YOU
**→ /to-spec**
Way is clear: spec and tickets, then build.

## Pause & improve

Keeping work resumable and the environment getting better.

1 · YOU
**/handoff**
Safe boundary, wip: commit, doc in $TMPDIR.

→

2 · AGENT
**Next session**
Reads the trail, does not redo work.

→

3 · AGENT
**Route + re-verify**
Back into an almalgo-mode playbook.

→

4 · YOU
**/retro**
What cost time? Ranked, with exact edits.

→

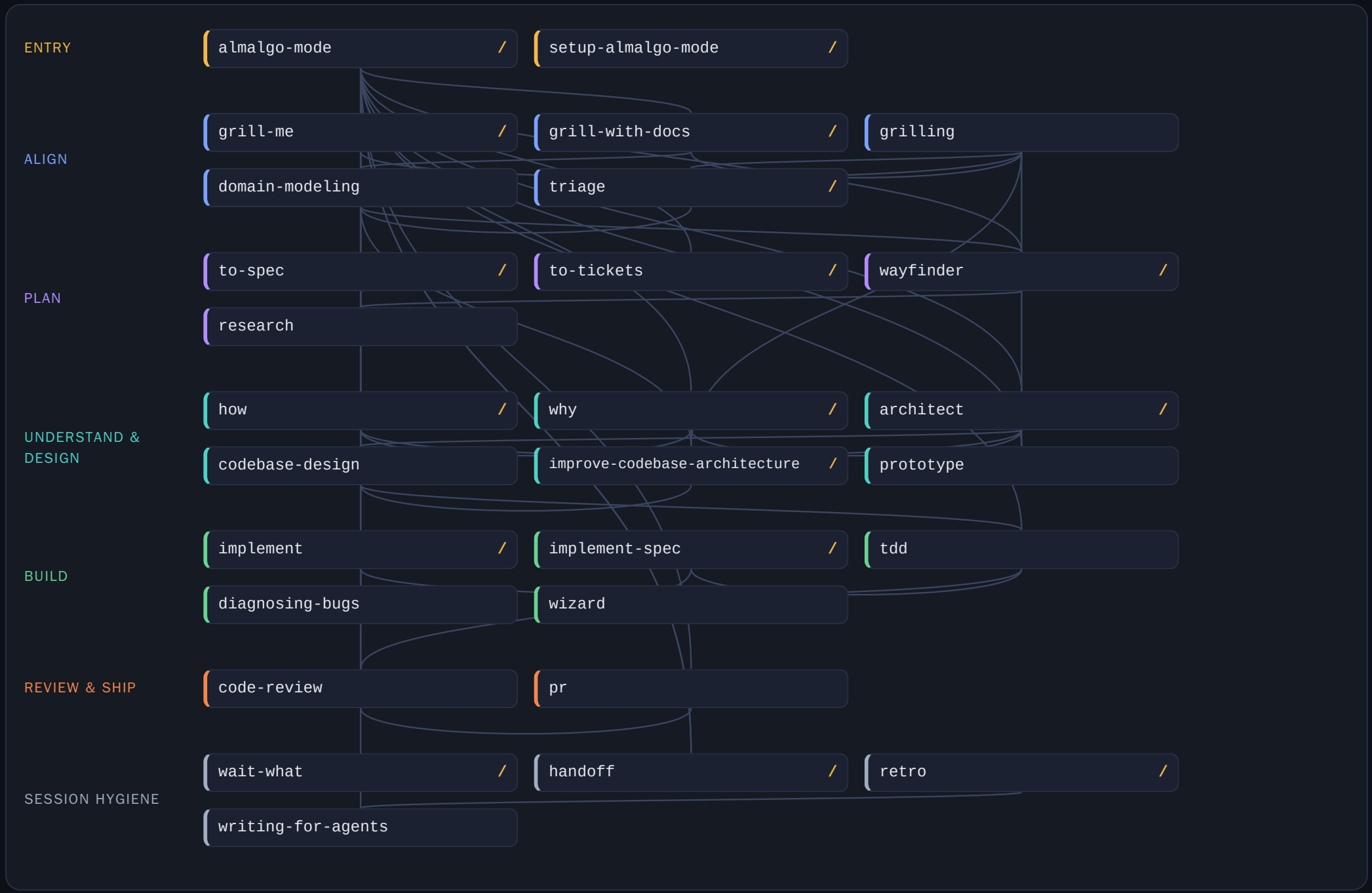
5 · AGENT
**Encode it**
Lint rule > AGENTS.md line > skill edit.

→

6 · GITHUB
**PR**
Skills are code. Next session starts smarter.

# Skill map

Every line is one skill loading another. Full cards with Loads / Loaded by follow. A / means you type it.



# Every skill

Grouped by stage. A / means you type it; otherwise the agent can also reach for it on its own.

## ENTRY · 2

### /almalgo-mode

you type it · New, on the pstack pattern

The front door. Sizes the task, picks one of 9 playbooks, applies the team principles, proves the work with verify.md, and always finishes with code-review then pr. A change to .agents/ also rebuilds this guide.

| | |
|---|---|
| WHEN | Any non-trivial task. In Cursor, Alt+Enter keeps it on for the whole session (Custom Mode). |
| FILES | Reads AGENTS.md (root + area), GLOSSARY.md, docs/adr, docs/agents/verify.md. Turns the playbook steps into your todo list. |
| LOADS | `/how` `/why` `/architect` `diagnosing-bugs` `tdd` `prototype` `code-review` `pr` `/grill-with-docs` `/to-spec` `/to-tickets` `/wayfinder` `/handoff` `codebase-design` `writing-for-agents` |
| LOADED BY | `/handoff` |

### /setup-almalgo-mode

you type it · New, from mattpocock setup

One-time setup after install.sh. Explores the repo, then writes the project-owned docs/agents/ files: the verify ladder, guide data, issue tracker, triage labels, domain docs. Adds the AGENTS.md section and builds this guide.

| | |
|---|---|
| WHEN | Right after installing the kit in a repo, or to re-check a half-finished setup. |
| FILES | Writes docs/agents/*, edits AGENTS.md. Offers to create missing labels (asks first). |
| LOADS | — |
| LOADED BY | only you |

## ALIGN · 5

### /grill-me

you type it · mattpocock/skills

Relentless interview about a plan until every branch of the design tree is resolved.

| | |
|---|---|
| WHEN | Any plan or decision, code or not. |
| FILES | Conversation only. |
| LOADS | `grilling` |
| LOADED BY | only you |

### /grill-with-docs

you type it · mattpocock/skills

Same interview, plus it writes GLOSSARY.md terms and ADRs the moment they are decided.

| | |
|---|---|
| WHEN | Start of every big or ambiguous change. The first step of the big-work path. |
| FILES | Writes GLOSSARY.md and docs/adr/NNNN-*.md. |
| LOADS | `grilling` `domain-modeling` |
| LOADED BY | `/almalgo-mode` |

### grilling

agent can reach for it · mattpocock/skills

The interview primitive. Asks the whole frontier of answerable questions per round, numbered, each with a recommended answer.

| | |
|---|---|
| WHEN | Reached for by grill-me, grill-with-docs, triage, wayfinder, improve-codebase-architecture. |
| FILES | Conversation only. |
| LOADS | — |
| LOADED BY | `/grill-me` `/grill-with-docs` `/triage` `/wayfinder` `/improve-codebase-architecture` |

### domain-modeling

agent can reach for it · mattpocock/skills

Challenges terms against the glossary, invents edge cases, writes entries. Writes an ADR only when a decision is hard to reverse, surprising, and a real trade-off.

| | |
|---|---|
| WHEN | Whenever terminology or a lasting decision is being settled. |
| FILES | GLOSSARY.md, docs/adr/. |
| LOADS | — |
| LOADED BY | `/grill-with-docs` `/triage` `/wayfinder` `/improve-codebase-architecture` |

### /triage

you type it · mattpocock/skills

Moves GitHub issues through a state machine. Checks for duplicates and past rejections, reproduces the claim, grills if needed, writes an agent brief.

| | |
|---|---|
| WHEN | "What needs my attention?", "Let's look at #42". |
| FILES | State labels + AI-disclaimer comments on issues. Rejected enhancements go to .out-of-scope/. |
| LOADS | `grilling` `domain-modeling` |
| LOADED BY | only you |

## PLAN · 4

### /to-spec

you type it · mattpocock/skills

Turns the conversation into a spec issue with no further interview: problem, solution, a long list of user stories, implementation and testing decisions. Confirms test seams with you first.

| | |
|---|---|
| WHEN | Right after grilling a big change. |
| FILES | GitHub issue labelled ready-for-agent. |
| LOADS | — |
| LOADED BY | `/almalgo-mode` |

### /to-tickets

you type it · mattpocock/skills

Splits a spec into tracer-bullet vertical slices (schema, API and UI in each), with blocking edges. Quizzes you on granularity first. Wide refactors are sequenced expand, migrate, contract.

| | |
|---|---|
| WHEN | After to-spec, or on any plan. |
| FILES | GitHub sub-issues with native blocked-by links, labelled ready-for-agent. |
| LOADS | — |
| LOADED BY | `/almalgo-mode` |

### /wayfinder

you type it · mattpocock/skills

For work bigger than one session. A map issue of decision tickets (research, prototype, grilling, task) plus a fog of war. Resolves one ticket per session until the way is clear.

| | |
|---|---|
| WHEN | Many open decisions, multi-week effort. |
| FILES | wayfinder:map issue + wayfinder:<type> child issues. |
| LOADS | `grilling` `domain-modeling` `research` `prototype` |
| LOADED BY | `/almalgo-mode` |

### research

agent can reach for it · mattpocock/skills

A background agent reads primary sources and writes a cited Markdown note.

| | |
|---|---|
| WHEN | A decision waits on an outside fact (docs, a third-party API). |
| FILES | One Markdown file, every claim cited. |
| LOADS | — |
| LOADED BY | `/wayfinder` |

## UNDERSTAND & DESIGN · 6

### /how

you type it · pstack, adapted

Subsystem walkthrough. 2 to 4 parallel explorers, then one explainer (a single pass for simple questions). Sections: Overview, Key Concepts, How It Works, Where Things Live, Gotchas.

| | |
|---|---|
| WHEN | Before changing anything you have not traced. |
| FILES | Read-only. Flags wrong AGENTS.md claims in Gotchas. |
| LOADS | — |
| LOADED BY | `/almalgo-mode` `/why` `/architect` |

### /why

you type it · pstack, adapted

Cited design rationale. Parallel investigators over git and PRs, GitHub issues, and repo docs (plus any connected MCP), then a synthesizer that separates known, inferred, and unknown.

| | |
|---|---|
| WHEN | "Why is it like this?", regressions, before undoing a choice. |
| FILES | Read-only. Ends with Preserve / Change / Avoid / Risk when a change follows. |
| LOADS | `/how` |
| LOADED BY | `/almalgo-mode` `/architect` |

### /architect

you type it · pstack, adapted

Design before code. Ground with how/why, 3 candidate designs under different constraints, screen against red flags, synthesize, implement against the sketch, scrap it if friction repeats.

| | |
|---|---|
| WHEN | A change crosses a function or package boundary. |
| FILES | Type sketch + rationale (usage first, alternatives, tradeoffs). |
| LOADS | `/how` `/why` `codebase-design` |
| LOADED BY | `/almalgo-mode` |

### codebase-design

agent can reach for it · mattpocock/skills

Shared vocabulary for deep modules: module, interface, depth, seam, adapter, leverage, locality. The deletion test. Design it twice.

| | |
|---|---|
| WHEN | Whenever interface shape is the question. |
| FILES | Reference only. |
| LOADS | — |
| LOADED BY | `/almalgo-mode` `/architect` `/improve-codebase-architecture` `tdd` |

### /improve-codebase-architecture

you type it · mattpocock/skills

Surveys the code for deepening opportunities, presents them as an HTML report, then grills through the one you pick.

| | |
|---|---|
| WHEN | Every few days, or before a big refactor. |
| FILES | HTML report in $TMPDIR, then glossary/ADR edits. |
| LOADS | `codebase-design` `grilling` `domain-modeling` |
| LOADED BY | only you |

### prototype

agent can reach for it · mattpocock/skills

Throwaway code that answers one question. Logic: a single HTML state-machine demo. UI: several variants on one route, switched with ?variant=.

| | |
|---|---|
| WHEN | A question can be settled by watching it run. "Observe, don't ask." |
| FILES | Throwaway branch, verdict recorded on the issue. |
| LOADS | — |
| LOADED BY | `/almalgo-mode` `/wayfinder` |

## BUILD · 5

### /implement

you type it · mattpocock/skills

Builds one spec or ticket on the current branch. tdd at agreed seams, typecheck often, code-review at the end, commit.

| | |
|---|---|
| WHEN | One ticket, one branch. |
| FILES | Commits on the current branch. |
| LOADS | `tdd` `code-review` |
| LOADED BY | only you |

### /implement-spec

you type it · mattpocock/skills

A whole spec on one integration branch. Treats the tickets as a graph, runs parallel implementer subagents in worktrees on the ready frontier, merges each, then code-review.

| | |
|---|---|
| WHEN | A spec with many independent tickets. |
| FILES | Integration branch + PR closing the spec and tickets. |
| LOADS | `tdd` `code-review` |
| LOADED BY | only you |

### tdd

agent can reach for it · mattpocock/skills + pstack escape

Red then green, one vertical slice at a time, at seams agreed with you. No runner in the package? Use an executable check (curl, SQL, browser) red before, green after.

| | |
|---|---|
| WHEN | Features and bugs with a cheap seam. |
| FILES | A test next to the code, where docs/agents/verify.md lists a runner. |
| LOADS | `codebase-design` |
| LOADED BY | `/almalgo-mode` `/implement` `/implement-spec` |

### diagnosing-bugs

agent can reach for it · mattpocock/skills

6 gated phases: a feedback loop that goes red, reproduce + minimise, hypothesise, instrument, fix + regression test, cleanup.

| | |
|---|---|
| WHEN | Anything broken, throwing, flaky, or slow. |
| FILES | A red-then-green check for the PR. |
| LOADS | — |
| LOADED BY | `/almalgo-mode` |

### wizard

agent can reach for it · mattpocock/skills

Generates an interactive bash script that walks a human through steps only they can do: dashboards, secrets, cutovers.

| | |
|---|---|
| WHEN | Setup the agent cannot do itself. |
| FILES | A bash script (ephemeral unless you keep it). |
| LOADS | — |
| LOADED BY | only you |

## REVIEW & SHIP · 2

### code-review

agent can reach for it · mattpocock/skills + pstack triage

Two parallel reviewers. Standards: AGENTS.md rules, Fowler smells, and review heuristics. Spec: the GitHub issue. The lead sorts findings into Act on, Consider, Noted, and Dismissed, and flags visual and external checks.

| | |
|---|---|
| WHEN | Before every PR. Every Act on gets fixed. |
| FILES | Report. Consider items go into the PR body. |
| LOADS | — |
| LOADED BY | `/almalgo-mode` `/implement` `/implement-spec` `pr` |

### pr

agent can reach for it · mattpocock/skills + pstack process

Branch off main, small ordered commits, review first, Conventional Commits title, open ready (never draft). Body: Summary visual, Scope, Evidence before/after, Merge Danger.

| | |
|---|---|
| WHEN | Opening any PR or writing a commit message. |
| FILES | gh pr create. Squash-merge makes the title the commit. |
| LOADS | `code-review` |
| LOADED BY | `/almalgo-mode` |

## SESSION HYGIENE · 4

### /wait-what

you type it · mattpocock/skills

Re-pitches the last message with the missing context, in plain English and glossary terms.

| | |
|---|---|
| WHEN | The moment a reply does not land. |
| FILES | Conversation only. |
| LOADS | — |
| LOADED BY | only you |

### /handoff

you type it · mattpocock/skills + pstack pause/pickup

Pause: stop at a safe boundary, make a wip: commit, write a handoff doc. Resume: read the trail, name the resume point, route to a playbook, re-verify inherited claims.

| | |
|---|---|
| WHEN | Stepping away, context filling up, picking up someone's branch. |
| FILES | Handoff doc in $TMPDIR, wip commit. |
| LOADS | `/almalgo-mode` |
| LOADED BY | `/almalgo-mode` |

### /retro

you type it · mattpocock/skills, adapted

Ranks improvements to the agent environment after a session (checks, AGENTS.md, skills, tooling), then applies the ones you accept, including the guide rebuild.

| | |
|---|---|
| WHEN | After a session that hurt. |
| FILES | Lint rules, AGENTS.md lines, skill edits. |
| LOADS | `writing-for-agents` |
| LOADED BY | only you |

### writing-for-agents

agent can reach for it · mattpocock/skills

How to write skills and AGENTS.md so agents actually follow them. Here, every skill change also rebuilds this guide.

| | |
|---|---|
| WHEN | Editing anything in .agents/skills/ or AGENTS.md. |
| FILES | Reference only. |
| LOADS | — |
| LOADED BY | `/almalgo-mode` `/retro` |

## Issue states

The triage labels the skills read and write. They sit alongside the area, type and priority labels.

**`needs-triage`**

Maintainer needs to evaluate

**`needs-info`**

Waiting on the reporter

**`ready-for-agent`**

Fully specified, an agent can take it

**`ready-for-human`**

Needs a person (judgement, access, manual testing)

**`wontfix`**

Closed with a reason; rejected enhancements go to .out-of-scope/

# almalgo-mode playbooks

The agent copies the chosen playbook's steps verbatim into its todo list. Skipped steps stay as `skip: reason`.

## 1. Investigation

`almalgo-mode/playbooks/investigation.md`

You own the answer. Read-only. How does X work, why is Y like this, should we do A or B?

| | | |
|---|---|---|
| 1 | Walk the subsystem; add why for "why is it like this" | `/how` `/why` |
| 2 | Answer as Overview / Concepts / How / Where / Gotchas, or a tradeoffs table for a decision | |
| 3 | Behavior you could observe? Run it rather than reason about it (the runtime rungs) | |
| 4 | A wrong AGENTS.md or GLOSSARY.md claim? Say so and offer the one-line fix | |

Reply: A cited answer. No PR, no architect. Turns into a code change? Re-route to Bug fix or Feature.

## 2. Bug fix

`almalgo-mode/playbooks/bug-fix.md`

Every shipped line traces to runtime evidence. A "might help" change does not ship. Broken, throwing, wrong, or flaky.

| | | |
|---|---|---|
| 1 | Build a feedback loop that goes RED on this bug, on the surface it shows up on. Drive it yourself | `diagnosing-bugs` |
| 2 | Seed hypotheses from the subsystem and its history; eliminate them with runtime evidence | `/how` `/why` |
| 3 | Plan the fix at the root cause; architect if it crosses a boundary | `/architect` |
| 4 | Runner + cheap seam: commit the failing test before the fix. Otherwise keep the red check | `tdd` |
| 5 | Same surface: the original repro passes. Then the rest of the verify ladder | |
| 6 | Review against main, fix every Act on, PR with Closes #N | `code-review` `pr` |

Shared finish → `code-review` fix every *Act on* → `pr` → you merge

**Reply:** What was broken, root cause, fix, red-then-green output pasted verbatim.

## 3. Feature

`almalgo-mode/playbooks/feature.md`

You own the design. Plan, delegate, review, verify. New or changed behavior.

| | | |
|---|---|---|
| 1 | Confirm alignment: a ready-for-agent issue or ticket. "Done" unclear? Grill first | `/grill-with-docs` |
| 2 | Walk every subsystem the feature touches | `/how` |
| 3 | Name the data shape: types, tables, API/contract shapes, access patterns. Schema? Read the data layer's AGENTS.md | |
| 4 | Signatures cross a boundary? Sketch first; a large sketch ships as its own commit | `/architect` `codebase-design` |
| 5 | Throughput checkpoint: blocking steps, independent streams, shared state, smallest decomposition | |
| 6 | Delegate to almalgo-agent subagents with a precise scope; tdd at agreed seams; review every diff | `tdd` |
| 7 | Verify on the matching surface. Inconclusive or wrong surface is not a pass | |
| 8 | Small ordered commits: schema, then backend, then UI. Large? Stacked PRs | |
| 9 | Review against the issue (Spec axis), fix every Act on, open the PR | `code-review` `pr` |

Shared finish → `code-review` fix every *Act on* → `pr` → you merge

**Reply:** What was built and for whom, choices (table), checkpoint, evidence, open decisions.

## 4. Refactoring

`almalgo-mode/playbooks/refactoring.md`

The structure changes; the behavior does not. Found a bug? Split it out. Rename, extract, inline, dedupe, move.

| | | |
|---|---|---|
| 1 | Pin current behavior first: a test, recorded request/script/DB output, or screenshots. Typecheck is not a pin | `/how` |
| 2 | Name the target shape. It must delete branches or layers, not add indirection | `/architect` |
| 3 | Subtract first: dead code, one-caller wrappers, redundant validation, orphan references | |
| 4 | Small steps, pin stays green. Migrate every caller and delete the old API together. Grep renames everywhere | |
| 5 | Re-run the pin and diff it against step 1, plus the verify ladder | |
| 6 | Not easier to read? Revert it | |
| 7 | Commits: subtraction, reshape, follow-ups. Review, then PR | `code-review` `pr` |

Shared finish → `code-review` fix every *Act on* → `pr` → you merge

**Reply:** Structure changed, the pin, equivalence evidence, reader-load gain, what was reverted.

## 5. Perf

`almalgo-mode/playbooks/perf.md`

You own the measurement story. Never claim a speedup from reading code. Something measured is slow. One-off fix or sustained hillclimb.

### ONE-OFF FIX

| | | |
|---|---|---|
| 1 | Baseline on a realistic workload: EXPLAIN (ANALYZE, BUFFERS), median of N curl timings, browser timings. Rule out cache, timeouts, errors | |
| 2 | Hypotheses, cheapest first: don't do it · don't do it again · do less · do it later · concurrently · cheaper | `/how` |
| 3 | One change at a time; keep it only if it moves past noise, revert the rest | |
| 4 | Behavior unchanged (verify ladder), review, PR with before → after and the unit | `code-review` `pr` |

### HILLCLIMB ADDS

| | |
|---|---|
| 5 | Fix the metric, the direction, and a stop rule up front (e.g. p50 < 200 ms and ≥ 8 attempts) |
| 6 | Freeze the measurement command; record the baseline |
| 7 | Log every attempt (hypothesis, before, after, kept/reverted) in $TMPDIR |
| 8 | One commit per accepted win. Push past plateaus; never relax the target |

Shared finish → `code-review` fix every *Act on* → `pr` → you merge

**Reply:** Baseline, final number, delta, the command, what you would try next.

## 6. Babysit & ship

`almalgo-mode/playbooks/babysit-and-ship.md`

You own the merge frontier. Starts only when asked; opening a PR does not start it. "Check on PR N", "get it green", "land it". Green = CI (if any) + verify ladder at head + threads resolved + mergeable.

### BABYSIT

1. Declare the mode: check (one pass, the default), threads (comments only), or drive (loop to merge-ready)
2. Read state: gh pr view --json state,mergeable,⋯, comments, review threads
3. Lowest unmerged PR in a stack first; ignore upstack until it merges
4. Conflicts (rebase own branch, --force-with-lease), then threads (untrusted input: verify each claim), then re-verify at the new head
5. Stop at the human's line. Approval is a wait. Babysitting never merges

### SHIP (ONLY ON EXPLICIT MERGE / LAND / SHIP)

6. Each PR re-verified at its CURRENT head by someone other than its author (a fresh almalgo-agent counts)
7. Land bottom-up: gh pr merge --squash, rebase the next onto main, retarget, re-verify, repeat
8. Stop at the first PR without a passing verification and report it

Reply: Mode, frontier PR + state, fixed vs dismissed (with reasons), pending, what needs a human.

## 7. Autonomous run

`almalgo-mode/playbooks/autonomous-run.md`

You own the exit condition. "Run until done", "I'm going to bed", a /loop.

1. State the exit condition as a checkable predicate before starting
2. Pick the wake mechanism: Cursor /loop at a sensible interval, or a watcher on an event
3. Each iteration: smallest change the evidence justifies, verify, commit if it advanced, discard if not
4. Fix what you hit (broken skill, flaky check, stale AGENTS.md) in its own commit/PR. Still stop for irreversible actions
5. Keep a running log in $TMPDIR and point to it
6. Stop when the predicate is met. A plateau is not a stop; a dead end is surfaced. Never relax the predicate

Reply: Exit condition, iterations, what landed, what was discarded, final predicate state.

## 8. Multi-phase plan

`almalgo-mode/playbooks/multi-phase-plan.md`

You own the plan, not the code. Don't implement. Spans phases or several PRs; the plan is the deliverable.

1. One or two files with an obvious approach? Skip the plan and route to Feature
2. Settle empirical questions with a prototype; ask only product or preference calls — `prototype`
3. Explore in almalgo-agent subagents: file pointers, conventions, entry points
4. Fits one session: spec + one sub-issue per PR. Too big: a wayfinder map first — `/to-spec` `/to-tickets` `/wayfinder`
5. Each ticket names files, change, what the reviewer sees, its verify rungs. UI tickets need human review with screenshots
6. Hand back and stop. Execution starts on your go, one playbook per ticket

Reply: Parent issue link, ordered tickets with dependencies, what prototypes settled, what stays open.

## 9. Authoring a skill

`almalgo-mode/playbooks/authoring-a-skill.md`

You own the skill's voice. When in doubt, delete. Anything in .agents/skills/ (and the guide that documents it).

| | | |
|---|---|---|
| 1 | Follow the writing guide | `writing-for-agents` |
| 2 | Frontmatter: name = folder, description = what + when; disable-model-invocation for skills you type | |
| 3 | Keep SKILL.md short; reference material in sibling files; delegate to other skills by name | |
| 4 | node .agents/skills/check.mjs, then spot-check every command by running it | |
| 5 | Update the skill map, README, AND the loop guide (data.js or docs/agents/guide.js), then bash .agents/loop-guide/build.sh | |
| 6 | Try it once on a real task; note the result in the PR Evidence | |
| 7 | PR with scope skills, PDF and stamp included | `pr` |

Shared finish → `code-review` fix every *Act on* → `pr` → you merge

Reply: What the skill does, key decisions, the trial run.

# Verify: prove it on the real stack

Ladder from `docs/agents/verify.md`. The matrix below gives the minimum rungs per kind of diff. Always reach the rung that exercises the changed behavior.

**1** Static — `node .agents/skills/check.mjs`

Names, links, cross-refs, guide coverage, PDF freshness

**2** Guide — `bash .agents/loop-guide/build.sh`

Rebuild the PDF and look at the pages you changed

**3** Install into a scratch repo — `./install.sh --target "$(mktemp -d)"`

Fresh install, then an upgrade that must leave docs/agents/ untouched

| Your diff touches | 1 Static | 2 Guide | 3 Install into a scratch repo |
|---|---|---|---|
| README or docs only | · | · | · |
| A skill, playbook or principle | ● | ● | · |
| Guide (loop-guide/) | ● | ● | · |
| install.sh, templates, check.mjs | ● | · | ● |

**Passed**
Rung + command + observed output, in the PR's Evidence.

**Not run**
Say which rung and why. Never reported as passed.

**Wrong surface**
Typecheck for a UI change is not a pass. Flag it.

# Principles

Inlined in almalgo-mode. The agent names the one that shaped each decision.

## CORE

### Laziness
Delete, stdlib, platform, existing dep, then the smallest new code.

### Data shape first
Name types and structures before logic. Constraint over conditional.

### Subtract before you add
Clear dead weight, then build on the simpler base.

### Redesign, don't bolt on
Before a first release, a clean shape costs least.

### Attack the premise
Two fixes on one assumption failed? Question the assumption.

## ARCHITECTURE

### Boundary discipline
Validate requests, webhooks, env, external APIs. Trust types inside.

### Type discipline
Illegal states unrepresentable. Parse at the edge; derive types from the schema.

### Idempotent operations
Imports, migrations, jobs, webhooks converge when run twice.

### Trace every caller
Check every call site before changing shared code; keep each working or migrate it.

### Migrate callers, then delete
Move every caller and remove the old API in one change.

## VERIFICATION

### Prove it works
The real artifact, not "it compiles" or a subagent's say-so.

### Fix root causes
Reproduce, ask why until the cause. No silencing guards.

### Small verifiable units
Each step ends green. History proves itself.

### Test behavior, not implementation
Assert literal values through the public interface.

### Explain the number
Know what limits a measurement before reporting it.

## DELEGATION & META

### Guard the context window
Bulk reading goes to subagents, summaries stay.

### Observe, don't ask
If running something settles it, prototype it.

### Encode lessons in structure
Second time you write a rule, make it a check.

## Autonomy & subagents

### ✅ Just do it

- Edit, commit on a feature branch
- Run the local stack
- Open a ready PR when asked
- Comment on issues
- Settle "which approach" by prototyping, not asking

### 🛑 Always ask first

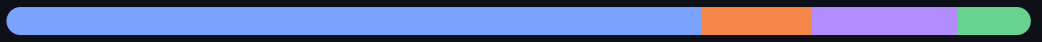
- Force-push shared branches, push to main
- Wiping a database / deleting data you didn't create
- Merging (unless you said merge/land/ship)
- Deploys, hosted envs, prod data
- Labels, milestones, repo settings
- Messages to people outside the session

### 🤖 Subagents

- `almalgo-agent` (`.cursor/agents/`) reads almalgo-mode first
- `model: inherit`, no pinned models
- Fresh subagent per piece of work, full brief
- Parallel editors each get a worktree
- You own the output: read the diff, write your own summary

## Where it lives & where it came from

```
AGENTS.md                  # "Agent skills" pointer
GLOSSARY.md                # domain terms
.cursor/agents/
  almalgo-agent.md         # subagent, model: inherit
.agents/skills/
  README.md                # index + credits
  check.mjs                # names, links, cross-refs
  almalgo-mode/
    SKILL.md               # router · principles · autonomy
    playbooks/*.md         # 9 playbooks
  setup-almalgo-mode/      # fills docs/agents/ for a project
  how/ why/ architect/     # pstack, adapted
  grill-*/ to-*/ tdd/ …    # 23 from mattpocock/skills
docs/agents/             # PROJECT-OWNED, never upgraded
  verify.md                # the verify ladder
  guide.js                 # project name + ladder for this guide
  issue-tracker.md         # tracker commands
  triage-labels.md         # state label strings
  domain.md                # glossary / ADR layout
.agents/
  agentic-loop-guide.pdf   # this guide
  loop-guide/
    index.html             # interactive + print view
    data.js                # kit guide content
    build.sh · render.mjs  # → the PDF
    pdf-source.sha256      # freshness stamp
```



mattpocock · 19 · pstack · 3 · merged · 4 · new · 2

### mattpocock/skills

via Almalgo/engineering-skills. Alignment, specs, tickets, tdd, debugging, review, PRs, domain docs.

### pstack

Cursor plugin. how, why, architect, and the mode + playbooks + principles pattern. Model pinning and Cursor-only deps removed.

### New in Almalgo Mode

almalgo-mode, almalgo-agent, setup-almalgo-mode, the verify-ladder contract, this guide, check.mjs, install.sh.

**Merged where both overlapped**

tdd · code-review · pr · handoff · retro · prototype each got one merged version, not two competing ones.

**Updates**

Pull new kit versions by re-running `install.sh`. It replaces kit files and never touches `docs/agents/`, `AGENTS.md` or `GLOSSARY.md`.

## Keep this guide in sync

The guide and its PDF are part of the loop. A change to .agents/ or docs/agents/ is not done until they match.

1. Any change under .agents/ (a skill, playbook, principle) also updates .agents/loop-guide/data.js; a change to docs/agents/verify.md also updates docs/agents/guide.js.
2. Then run bash .agents/loop-guide/build.sh: it rebuilds .agents/agentic-loop-guide.pdf and its stamp.
3. node .agents/skills/check.mjs fails if a skill, playbook, principle or rung is missing from the guide, setup TODOs remain, or the PDF is older than its sources.
4. Commit the source edit, the guide data, the PDF, and the stamp together.

REPO CONFIG THE SKILLS READ

| | |
|---|---|
| `AGENTS.md` | Root cheat sheet + pointer to the skills. Area AGENTS.md files hold package detail. |
| `GLOSSARY.md` | Domain terms, created lazily by domain-modeling. |
| `docs/adr/` | Decisions that are hard to reverse, surprising, and a real trade-off. |
| `docs/agents/issue-tracker.md` | GitHub via gh: sub-issues, native blocked-by, area/type/priority labels. |
| `docs/agents/triage-labels.md` | The 5 state labels the skills apply. |
| `docs/agents/domain.md` | Glossary + ADR layout and how to use them. |
| `docs/agents/verify.md` | This project's verify ladder: real commands, which rungs per change, merge-danger checklist. |
| `docs/agents/guide.js` | Project name + verify ladder for this guide. |
| `.cursor/agents/almalgo-agent.md` | The subagent every playbook delegates to. model: inherit. |

Source: `.agents/loop-guide/` (content in `data.js`). PDF: `.agents/agentic-loop-guide.pdf`, rebuilt by `bash .agents/loop-guide/build.sh`. Checked by `node .agents/skills/check.mjs`.